Análisis sintáctico

También se conoce como "Parsing", es la segunda etapa de un compilador que se realiza después del análisis léxico. En esta primera etapa, el codigo fuente es fragmentado en piezas pequeñas llamadas TOKENS. El análisis sintáctico revisa como esos TOKENS son ordenados.

El objetivo principal del análisis sintáctico es asegurar que los TOKENS esten ordenados de forma correcta y coherente, de acuerdo con las reglas gramaticales del lenguaje de programación. Si la estructura es correcta, el analizador crea un árbol de sintaxis abstracta (AST), que muestra la estructura del programa de forma clara y jerárquica, y ayuda al compilador a comprender mejor el código

FUNCIÓN

- Entrada: los TOKENS que produce el analizador léxico.
- Verifica la estructura gramatical del programa.
- Detecta errores de sintáxis, como un punto y coma faltante o paréntesis sin cerrar.
- Construye el árbol sintáctico o AST para fases posteriores.

CONCEPTOS CLAVE

- Gramática libre de contexto (GLC): conjunto de reglas que define que secuencia de TOKENS son validas.
- Derivación: Aplicación paso a paso de las reglas, desde el símbolo inicial, hasta generar la cadena de de TOKENS (base para construir el árbol de análisis).
- Árbol de Sintaxis Concreta (CTS): representa la estructura gramatical completa, con todos los símbolos y la puntuación.
- Árbol de Sintaxis Abstracta (AST): Una versión simplificada del CTS que solo conserva lo esencial. Se prefiere en fases posteriores porque es más fácil de analizar y optimizar.
- Ambigüedad: Confunde al compilador.

CARACTERISTICAS

- Construcción de AST muestra como se relacionan sentencias, expresiones y operadores.
- Validación basada en la gramática con las reglas GLC
- Técnicas de análisis:
  - Descendente (top-down): Construye de la raíz hacia las hojas
  - Ascendente (bottom-up): Construye de las hojas hacia la raíz
- Detección de errores: Indicado en que linea para hacer correcciones.

PASOS EN LA FASE DE ANÁLISIS SINTACTICO

1. Análisis: compara los TOKENS con la gramatica y, si son validos, construye el árbol AST.
2. Manejo de errores: detecta los errores de sintaxis y genera mensajes claros que indican su ubicación.
3. Interacción con la tabla de símbolos: registra y actualiza información de los identificadores, como su nombre y ámbito, que despues usan otras fases del compilador.